

UNIVERSIDADE DE SÃO PAULO

**PEA - DEPARTAMENTO DE ENGENHARIA
ENERGIA E AUTOMAÇÃO ELÉTRICAS**

**CONTROLADORES LÓGICOS
PROGRAMÁVEIS**

Prof. Dr. SERGIO LUIZ PEREIRA

2003

1. OBJETIVOS:

Esta apostila tem como principal objetivo apresentar ao aluno os tópicos fundamentais sobre Controladores Programáveis CPs ou Controladores Lógicos Programáveis CLPs como:

- Histórico
- Arquitetura de hardware
- Ciclo de operações
- Memórias, endereçamento e instruções de BIT
- Aplicações

Esta apostila também apresenta um resumo do sistema de endereçamento do CLP SLC-500 e do sistema de programação e de operação dos softwares RS LINX e RS LOGIX.

(Obs. Tanto o CLP SLC-500 como os softwares RS LINX e RS LOGIX são produtos da Rockwell Automation doados a Universidade de São Paulo por meio do convênio EPUSP/Rockwell Automation do Brasil.)

2. INTRODUÇÃO HISTÓRICO DOS CLPs:

O desenvolvimento do CLPs (Controladores Lógicos Programáveis) ou CPs (Controladores Programáveis) começou por volta de 1968 quando a General Motors solicitou à indústria eletrônica uma alternativa para a lógica eletromecânica baseada em relés.

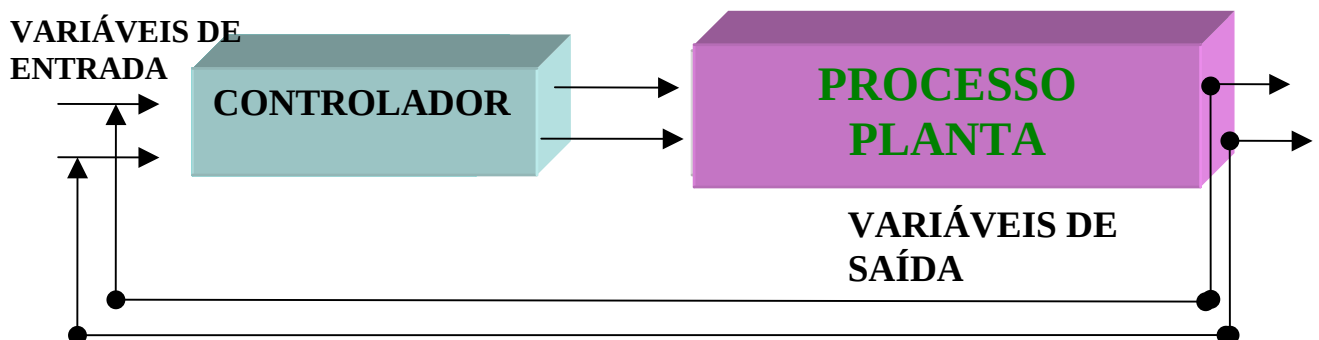


Fig. 1 Diagrama em blocos de um sistema de controle genérico.

Os sistemas de relés utilizados nas atividades industriais: montagem, carregamento e controle de máquinas haviam se tornados grandes e complexos, aumentando significativamente os custos de manutenção e baixando a confiabilidade.

Outro problema era a grande complexidade envolvida em qualquer mudança na planta industrial ou produtiva.

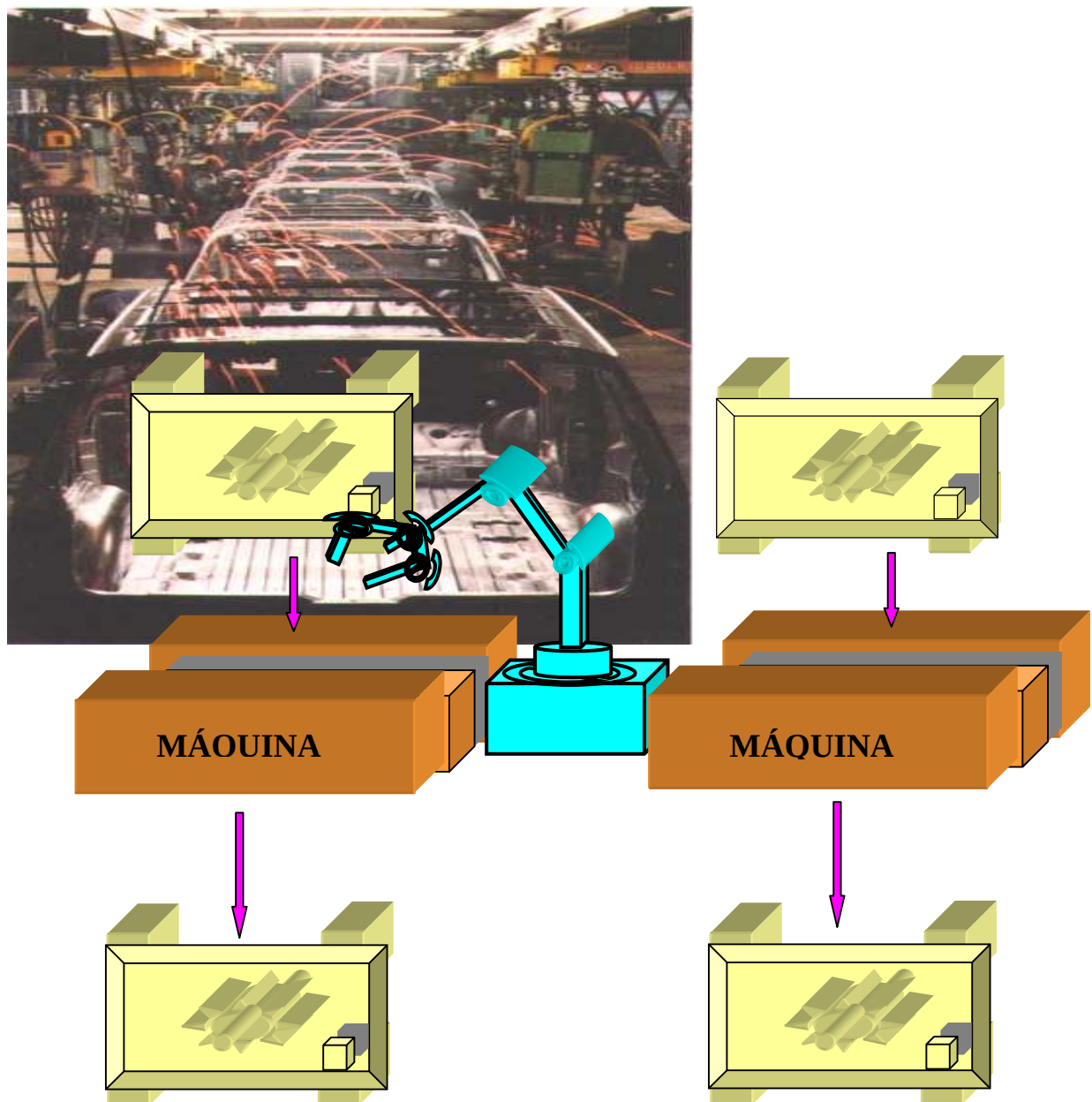


Fig. 2 Ilustrativo de uma planta industrial

Inicialmente os CLPs foram produzidos somente para simular a ação de relés num circuito de intertravamento. Hoje, os CPs também incorporam funções avançadas como: controle estatístico, controle de malha, comunicação em rede etc.

3. ARQUITETURA DOS CLPs

O CLP é um equipamento de estado sólido que pode ser programado para executar instruções que controlam dispositivos, máquinas e operações de processos, por meio da implementação de funções específicas como lógica de controle, sequenciamento, controle de tempo, operações aritméticas, controle estatístico, controle de malha, transmissão de dados, etc.

Os CLPs são projetados e construídos para operarem em ambientes severos, portanto devem resistir a altas temperaturas, ruídos elétricos, poluição atmosférica, ambientes únicos etc.

Sua capacidade quanto ao número de entradas e saídas, memória, conjunto de instruções, velocidade de processamento, conectividade, flexibilidade, IHM etc. varia conforme o fabricante e modelo.

A figura seguinte ilustra a arquitetura básica de um CLP genérico composto de CPU, memória, dispositivos de entrada e de saída, interface homem/máquina e fonte controlando uma planta industrial genérica.

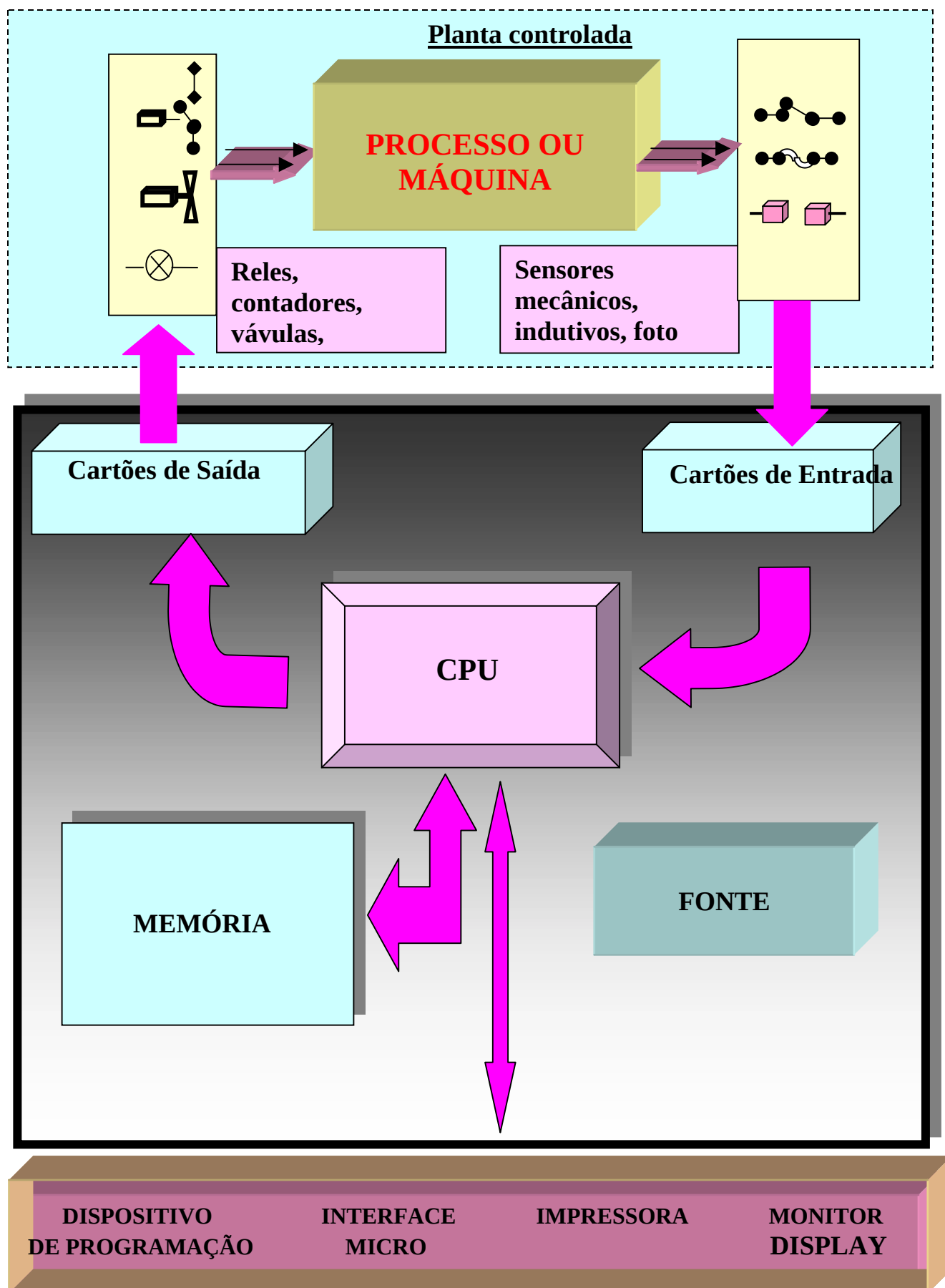


Fig. 3 Ilustrativo da arquitetura básica de um CLP.

A arquitetura básica de um CLP é composta de cinco elementos:

a – Unidade Central de Processamento (CPU). A CPU é o centro nervoso do sistema. Ela recebe os sinais digitais e os sinais analógicos dos sensores do campo conectados aos módulos de entrada e também recebe os comandos e os dados via comunicação em rede (quando for o caso). Em seguida executa as operações lógicas, as operações aritméticas e avançadas como as de controle de malha programadas na memória do usuário e atualiza os cartões de saída digitais e analógicos.

b – Memória.

É dividida em duas partes: instruções do programa executivo que controla as atividades da CPU e instruções do programa de aplicação do usuário. Normalmente esta última memória pode ser expandida pelo usuário.

c – Módulos de entrada e saída (E/S) também chamados de Cartões de I/O

Possibilitam a interface entre os dispositivos do usuário com a CPU. Existe uma grande variedade de módulos de entrada e de saída como: módulo de entrada de corrente contínua para tensões de 24 Volts., módulo de entrada de corrente alternada para tensões de 220 Volts., módulo de entrada analógico de tensão e corrente, módulo de saída analógico de tensão ou de corrente, módulos especiais para controle de malha, módulos de comunicação, etc.

d – Dispositivos de Programação e de Leitura (HIM).

São os diversos dispositivos de interface Homem/máquina conectados aos CLPs. Atualmente os dispositivos de programação dos CLPs também variam em tamanho e capacidade. Também podem servir para monitorar o andamento do programa, as variáveis internas e os dispositivos de campo. Podem ser portáteis ou não, e também são empregados para a introdução do programa de aplicação na memória do CLPs. Muitos fabricantes fornecem ou vendem pacotes de software, para que a programação e edição seja feita em um micro tipo PC para que o programa depois de editado no micro seja carregado no CLP diretamente ou por meio de uma rede de comunicação.

e – Fonte de alimentação.

Fornece todos os níveis de tensão exigidos para as operações internas do CLP. Convém lembrar que como geralmente os CLPs são modulares, existem casos que uma segunda fonte é necessária devido ao aumento de consumo com a expansão dos módulos de saída. Cada fabricante especifica às condições que tornam necessárias uma segunda fonte. Certos modelos de CLPs são projetados para operarem com uma tensão de alimentação de 220 Volts., outros trabalham com tensão de alimentação contínua de 24 Volts.

4. PRINCÍPIO DE FUNCIONAMENTO E CONCEITO DE SCAN

O princípio de funcionamento de um CLP baseia-se no ciclo denominado SCAN.

A CPU de um CLP processa as instruções que são programadas pelo usuário e juntamente com a realimentação da situação (“status”) dos dispositivos de entrada e de saída, gera os comandos para os módulos de saída. Os comandos enviados controlam dispositivos como: reles, contadores, solenóides, lâmpadas, etc. A figura seguinte ilustra o SCAN de um CLP.

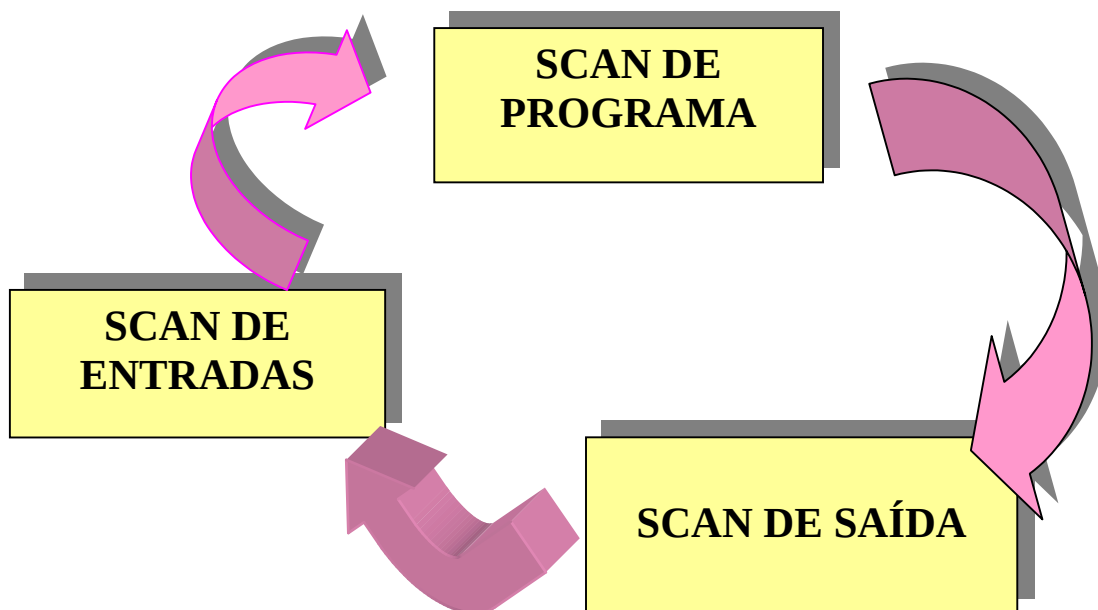


Fig. 4 Ilustrativo do ciclo de SCAN de um CLP.

O SCAN de um CLP é sub-dividido em três SCANS: SCAN de entrada, SCAN de programa e SCAN de saída. Durante o SCAN de entrada é efetuada a leitura de todas as variáveis e dados disponíveis nos cartões de entrada. Ou seja, é durante o SCAN de entrada que os dados disponíveis nos cartões de entrada são copiados para uma área de memória da RAM geralmente definida como área de imagem de entrada (endereço da memória pré-estabelecido).

Terminado o SCAN de entrada a CPU inicia o SACN de programa. de entrada no “SCAN de entrada”, em seguida, inicia a seqüência do programa efetuado pelo usuário. É durante o SACN de programa que a lógica programada pelo usuário é executada.

Terminado o SACN de programa é iniciado o SCAN de saída quando então os cartões de saída serão atualizados com os dados ou variáveis que estão na área de memória RAM geralmente definida como imagem das saídas de um CLP, (endereço de memória pré-estabelecido) .

Diversos modelos também apresentam a possibilidade de se alterar o ciclo do SCAN gerando dessa forma quatro tipos básicos de SCAN:

A – Ciclo elementar (entradas→programa→saídas).

B - Ciclo de entradas agrupadas, saídas distribuídas)
(entradas→processamento→saídas→processamento de saídas).

C - Ciclo distribuído (E/S ativadas conforme requisitadas)
(E→P→O→I→P→.....

D – Ciclo distribuído com sub “clock” .

Existe hoje no mercado uma grande variedade de linguagens para programação de CLPs, porém, a linguagem até hoje mais utilizada é a linguagem “Relay Ladder Diagrams”, comumente chamada de “linguagem de ladder” combinada com “funcional blocks”. Nesta linguagem os comandos imitam a estrutura de um esquema de circuito de intertravamento baseado em lógica de relés. Entretanto convém lembrar que esta estrutura de linguagem assemelha-se mas, não opera exatamente como um circuito de relés.

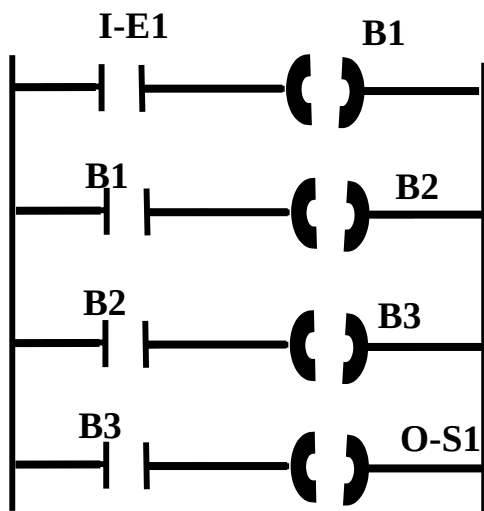
Um programa de CLP é composto basicamente de instruções de entrada e instruções de saída. Durante uma instrução de entrada a CPU verifica uma pergunta ou uma comparação. Caso a resposta seja afirmativa é estipulado durante o SACN atual

uma continuidade lógica do trecho de linha. Uma instrução de saída é processada conforme exista ou não continuidade lógica de linha até ela.

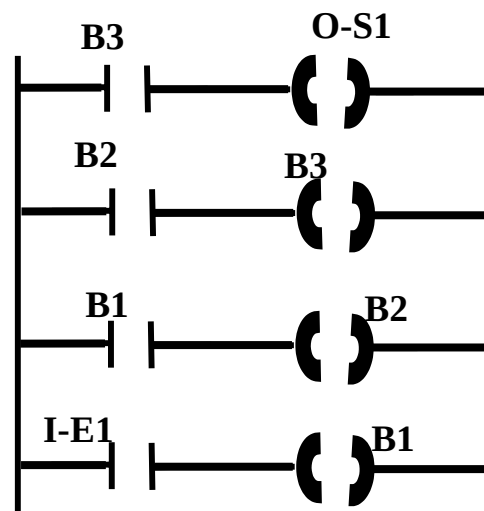
O tempo de SCAN total depende da velocidade (potência) do CLP e do tamanho do programa de aplicação do usuário. Neste tempo deve ser computado o tempo de atualização dos módulos de entrada e saída. Determinadas instruções dos CLPs por serem mais complexas necessitam de um tempo maior de processamento. Os valores típicos de SCANS dos CLPs comerciais variam de 1 ms até 10 ms para cada 1 k típico de instruções. Portanto deve conhecer antecipadamente qual o período de ciclo do processo que se deseja controlar utilizando um CLP, e se o tempo do scan do CLP é significativamente menor para se atingir as condições básicas de controlabilidade. Esses dados são importantes tanto para os processos de intertravamento, como para os processos de controle contínuo de malhas fechadas, quando o CLP utiliza instruções ou módulos de controle PID (Proporcional, Integral, Derivativo).

No SCAN de programa as instruções do programa são executadas sequencialmente (ou seja, uma instrução depois da outra na seqüência direta de como foram carregadas no programa), portanto a ordem de como elas são introduzidas no programa pode alterar o número de SCAN necessários para se obter uma determinada condição ou até mesmo a própria condição final de uma determinada lógica.

As figuras seguintes ilustram como no número de SCANS necessários para que uma variável de saída seja alterada em função de uma alteração de uma variável de entrada pode ser maior em função de como o programa foi editado e como o resultado também pode ser outro.



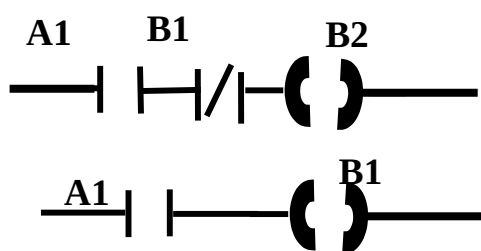
Se I-E1 for verdadeiro,
A Saída conectada à O-S1
Será acionada após um
Scan.



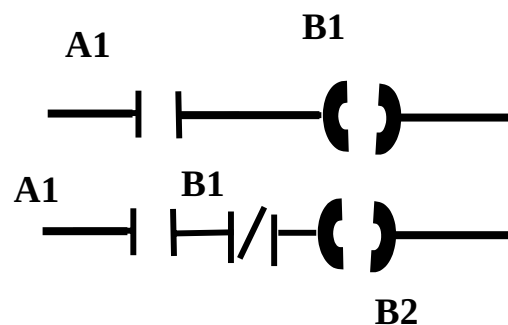
Se I-E1 for verdadeiro,
A Saída conectada à O-S1
Será acionada após quatro
Scan.

Fig. 5 Ilustrativo de alteração de performance em função da seqüência de instruções introduzida.

A figura seguinte ilustra um exemplo de alteração de resultado em função da seqüência de programação.



Sempre que A1 for verdadeiro
B2 ficará verdadeiro no presente
scan, e será desligado
Do segundo scan em diante.



B2 nunca ficará verdadeiro.

Fig. 5 Ilustrativo de alteração de performance em função da seqüência de instruções introduzida.

Existem basicamente cinco tipos de linguagens de programação para os CLPs .

- a - “Ladder Diagrams”
- b - “Functional Blocks”
- c - “Boolean Mnemonics”
- d - “English Statements”
- e – “GRAFSET” ou Sequential Function Chart (SFC)

Dependendo do modelo, o CLP pode aceitar duas ou mais linguagens de programação. Existem CLPs cuja linguagem de programação aceita é formada pela combinação entre dois tipos. Normalmente a combinação mais comum é “Ladder Diagrams” com “Functional Blocks”.

Entretanto, dependendo do modelo e do fabricante do CLP, o set de instruções pode conter instruções avançadas como por exemplo funções de controle de malha por algoritmo PID e por lógica Fuzzy.

5. INSTRUÇÕES BÁSICAS EM DIAGRAMA LADDER (CLPs)

- **Instruções de BIT**

Examinar

- (XIC) → **Examinar se energizado**
- (XIO) → **Examinar se desenergizado**
- (OSR) → **Monoestável sensível à Borda de Subida**
- (OTE) → **Energizar saída**
- (OTL) → **Energizar saída com retenção**
- (OTU) → **Desabilitar saída com retenção**

ARQUIVOS QUE PODEM UTILIZAR

Arquivo de Entrada e Saída

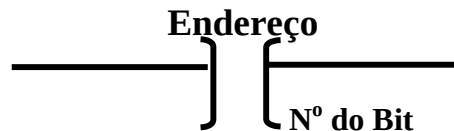
Arquivo de Status

Arquivo de BIT

Arquivo de temporizador, contador e controle

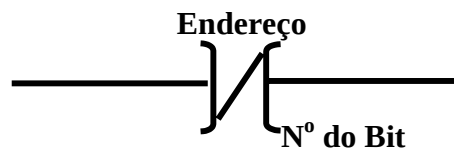
Arquivo Inteiro

- XIC



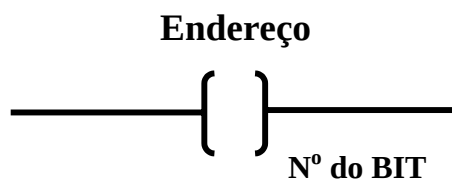
Estado do BIT	Instrução XIC
0	Falsa
1	Verdadeira

- XIO

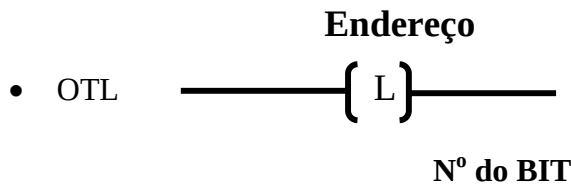


Estado do BIT	Instrução XIO
0	Verdadeira
1	Falsa

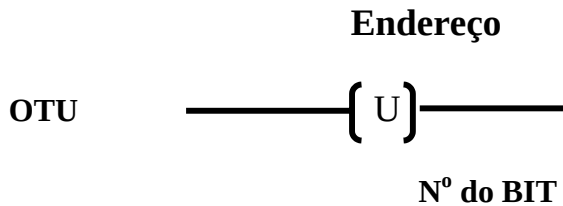
- OTE



Caso haja continuidade lógica da linha o bit endereçado pela instrução será colocado no estado lógico 1. Se não houver continuidade na linha o bit endereçado pela instrução será colocado no estado lógico 0.



Uma vez habilitada a saída endereçada pela instrução, a mesma somente será desabilitada caso a instrução OTU seja acionada.



Desabilita uma saída habilitada por uma instrução OTL.

INSTRUÇÕES: TEMPORIZADOR E CONTADOR

- TON → Temporizador → **utiliza 3 palavras**
- (Exemplo de instrução temporizador)

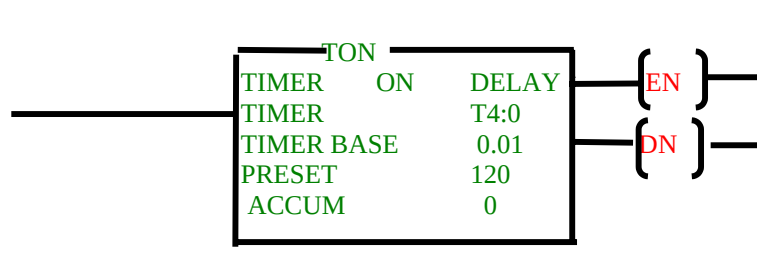
15	14	13	
EN	TT	DN	USO INTEANO
VALOR PRÉ SELECIONADO			
VALOR ACUMULADO			

EM (15) → **BIT de Habilitação**

TT (14) → **BIT de Temporização**

DN (13) → **BIT de Executado**

(Exemplo da instrução temporizador em diagrama ladder)



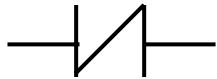
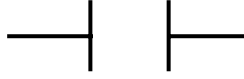
Inicia a contagem nos intervalos da base de tempo selecionada quando a condição da linha se torna verdadeira.

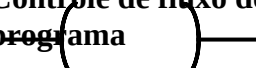
- TOF → Iniciar a temporização quando a condição da linha realiza uma transição de falsa para verdadeira. Funciona de maneiras contrárias ao TON
 - RTO → Igual ao TON com a diferença que o valor acumulado é retido quando a linha torna-se falsa.
 - RES → Zera o valor acumulado do contador
 - CTU → Contador crescente
 - CTO → Contador decrescente
 - RES → Zera o (CTU e CTO)
- } Incrementam o valor acumulado a cada transição do F/V

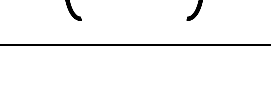
(Exemplo)

								09	08	07	06	05	04	03	02	01	00
CU	CD	DN	OV	UM	VA	Não Utilizada											
PRÊ-SEL.																	
ACUM.																	

As tabelas seguintes ilustram o set básico de um CP genérico.

Tipo de Operação	Linguagem Básica	Linguagem de Alto Nível
Linguagem Lógica        	L	Contato normalmente aberto. Contato normalmente fechado. Bobina de Energização. Bobina de Est. Complementar de Energização. Bobina com Memória ao Est./ON. Bobina de Desativação da Anterior Contato Transacional OFF-ON. Contato Transacional ON-OFF.
Temporização e Contadores       	TON TOF RTO RTR CTU CTD CTR	Temporizador ao fechamento. Temporizador a abertura. Temporizador ao fechamento com memória. Bobina de Desativação dos Temp. Retentivos. Contador Incremental crescente. Contador Incremental decrescente Bobina de desativação dos contadores.

Aritmética    	 + - x ÷	 Soma Subtração Multiplicação Bobina de Desativação dos Temp. Retentivos.
Manipulação de dados   	 CMP = CMP > CMP <	 Comparação de igualdade Comparação maior que Comparação menor que
Transferência de dados  	 GET PUT	 Leitura de conteúdo Escrita de conteúdo
Controle de fluxo de programa  	 MCR ZCL	 Relé mestre Relé de controle de último estado

	JMP	Desvio condicional
	LBL	Endereço do desvio condicional
	JSB	Chamada de sub-rotina
	RET	Retorno de sub-rotina

A tabela seguinte ilustra a equivalência entre os dois tipos de linguagem:

MNEMONIC	FUNCTION	LADDER EQUIVALENT
LD/STR	Load/Start	!-] [-
LD/STR NOT	Load/Star Not	!-]/[-
AND	And Point	-] [-
AND NOT	And Not Point	-]/[-
OR	Or Point	!
OR NOT	Or Not Point	!
OUT	Energize Coll	—()—
OUT NOT	De-Energize Coil	—(/)—
OUT CR	Energize Internal Coll	—()—
OUT L	Latch Output Coll	—(L)—
OUT U	Unlatch Output Coll	—(U)—
TIM	Timer	—(TON)—
CNT	Up Counter	—(CTU)—
ADD	Addition	—(+)—
SUB	Subtraction	—(-)—
MUL	Multiplication	—(x)—
DIV	Division	—(÷)—
CMP	Compare =, <, >	—(CMP)—
JMP	Jump	—(JMP)—
MCR	Master Control Relay	—(MCR)—
END	End MCR, Jump, or Program	—(END)—
ENT	Enter Value for Register	No Required

6 OUTRAS LINGUAGENS DE PROGRAMAÇÃO DE CLPs

Em 1975, a Association Française pour la Cybernétique Económiue et Technique (AFCET), formou uma comissão de trabalho chamada “*Comission de Normalisation de la Représentation du Cahier de Charges d’un Automatisme Lógique*”, com objetivo de conceber uma ferramenta única para especificação de sistemas de controle lógicos.

Esta mesma comissão apresentou em 1978 um comunicado, propondo um modelo para a descrição e representação da especificação de um sistema lógico complexo, batizado de GRAFCET (Graphe de Comando Etape-Transition).

A principal característica dessa metodologia é que os métodos de análise propostos para redes de Petri-Net podem ser utilizados na descrição e representação de sistemas.

GRAFCET também é chamado de Sequential Function Chart (SFC), e algumas grandes companhias do mercado a vêm incluindo em seus modelos, como no caso da Allen-Bradley na família PLC-5, que tanto pode ser programada por Ladder Logic Diagrams ou por SFC.

Uma companhia que também teve sucesso com uma diferente linguagem de programação de CPs é a Adatek of Sandponit, Idaho. Adatek desenvolveu em 1980 uma linguagem chamada Process State Monitor (PSM), e já possui mais de 1300 sistemas instalados no mercado Americano. PSM é uma linguagem de alto nível tipo “*state-type*”, que permite ao programador especificar diretamente o que deve acontecer com o sistema. Consiste apenas de 12 caracteres ASCII, que podem ser utilizados em testamentos que descrevem o sistema. O diagrama de estados é primeiramente construído por meio de blocos que mostram os estados do sistema, as funções, as condições e as transições de um estado para o outro.

Como nos últimos anos uma grande variedade de novos modelos de CPs foi introduzida no mercado, bem como várias linguagens novas de programação, alguns autores prevêm que a linguagem Ladder será aposentada em breve. Contudo, esta é uma questão difícil, pois como se sabe, existe um número significativamente grande de usuários e programadores já familiarizados com a linguagem de relés.

7 ENDEREÇAMENTO E SISTEMAS DE NUMERAÇÃO TRADICIONAIS DE CLPs

A capacidade de memória e a filosofia de endereçamento dos CLPs varia de modelo para modelo e de fabricante para fabricante. Porém, qualquer CLP deve ter uma área de sua tabela de dados que represente uma imagem virtual das entradas ligadas aos cartões de entrada, e uma área da tabela de dados que represente uma imagem virtual das saídas ligadas aos cartões de saída. Como geralmente os CLPs são moduladores, (ou seja a sua configuração pode ser expandida dentro de certos limites) estas áreas podem também variar respeitando a filosofia de projeto do fabricante. Essas áreas são normalmente designadas como imagem das entradas e imagem das saídas.

Alguns fabricantes especificam quais são os endereços válidos para cada tipo de função, outros permitem uma maior flexibilidade. Porém qualquer que seja o modelo e sistema de numeração empregado no endereçamento, a filosofia dos diversos CLPs é parecida.

Normalmente os CPs utilizam palavras de 16 bits chamadas de registradores ou registros e variáveis binárias de um bit chamados de pontos. Entretanto, já existem no mercado CLPs que utilizam palavras de 32 bits.

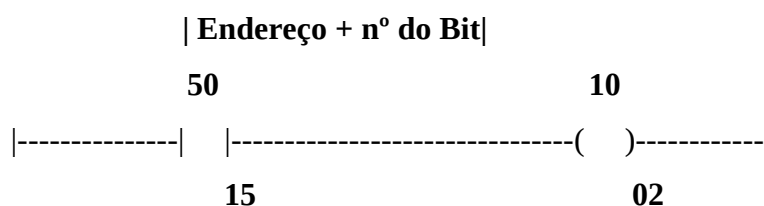
As variáveis binárias são utilizadas para representar os pontos de entrada e saída (contatos) e também para representar bobinas internas que podem ser: bobinas retentivas, bobinas transicionais, bobinas globais, bobinas de controle, etc.

A figura seguinte ilustra uma parcela da área de memória de um CLP genérico.

																bits	
Endereços	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	

Fig. 6 Área de memória de um CLP genérico

Assim sendo, a codificação em Diagrama Ladder de um endereço de um CLP genérico fica.



O exemplo acima significa que o registro número (endereço) 50 bit 15 se for levado para condição 1 (verdadeiro) acionará a bobina representada pelo registro número (endereço) 10 bit 02. Portanto, ao olhar para o número do Ponto, o usuário pode determinar o registrador que contém o número do ponto e o bit individual dentro daquele registrador.

Caso deseje-se registrar valores de temporizações, contagens, valores estatísticos, etc., então por exemplo, os registradores serão tidos nos sistemas 16 bits como variáveis de 16 bits podendo armazenar valores entre +32768 a -32767.

8 CONTROLE DE MALHA FECHADA, MONITORIZAÇÕES E INTEGRAÇÃO DE CLPs A UM SISTEMA DE CONTROLE HIERARQUIZADO.

Alguns modelos de CLPs permitem controle de malha em processos contínuos. Neste caso além dos cartões convencionais de E/S, eles podem ser ligados a cartões analógicos de leitura (A/D) e escrita (D/A). Uma configuração típica de controle é ilustrada na figura seguinte.

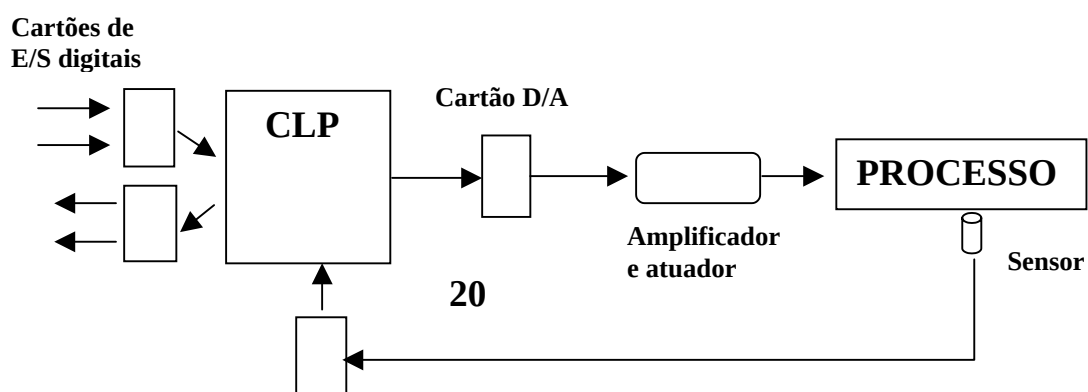


Fig. 7 Ilustrativo de um CLP sendo também empregado em um controle de malha.

9.1 Aplicações

Inicialmente (anos 70) os CLPs foram concebidos para trabalharem isoladamente no controle de processos. Porém, com a evolução dos demais equipamentos de controle destinados a automação e a necessidade de uma integração de diversos processos produtivos, os fabricantes de CLPs vem desenvolvendo novos modelos que possibilitam a integração dos mesmos as sistemas supervisionados e hierarquizados. Hoje existem redes de CLPs e de automação que permitem a sua integração dentro dos padrões (OSI), podendo dessa forma estarem integrados a um (SDCDs) por exemplo, ou trabalharem dentro de uma estrutura (CIM). É comum encontrar CLPs integrados a robôs e máquinas de CNCs formando células flexíveis de montagens.

A figura seguinte ilustra o conceito da pirâmide da automação.

NÍVEIS

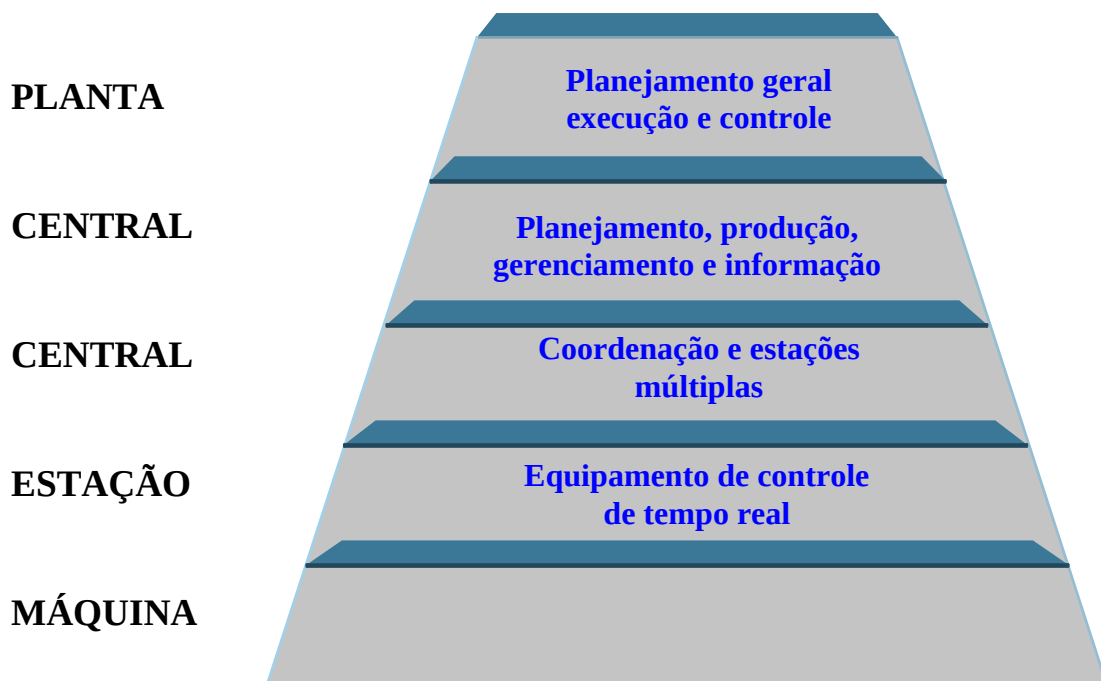


Fig. 8 Ilustrativo da pirâmide industrial.

A figura seguinte ilustra uma automação efetuada até o nível três da pirâmide industrial.

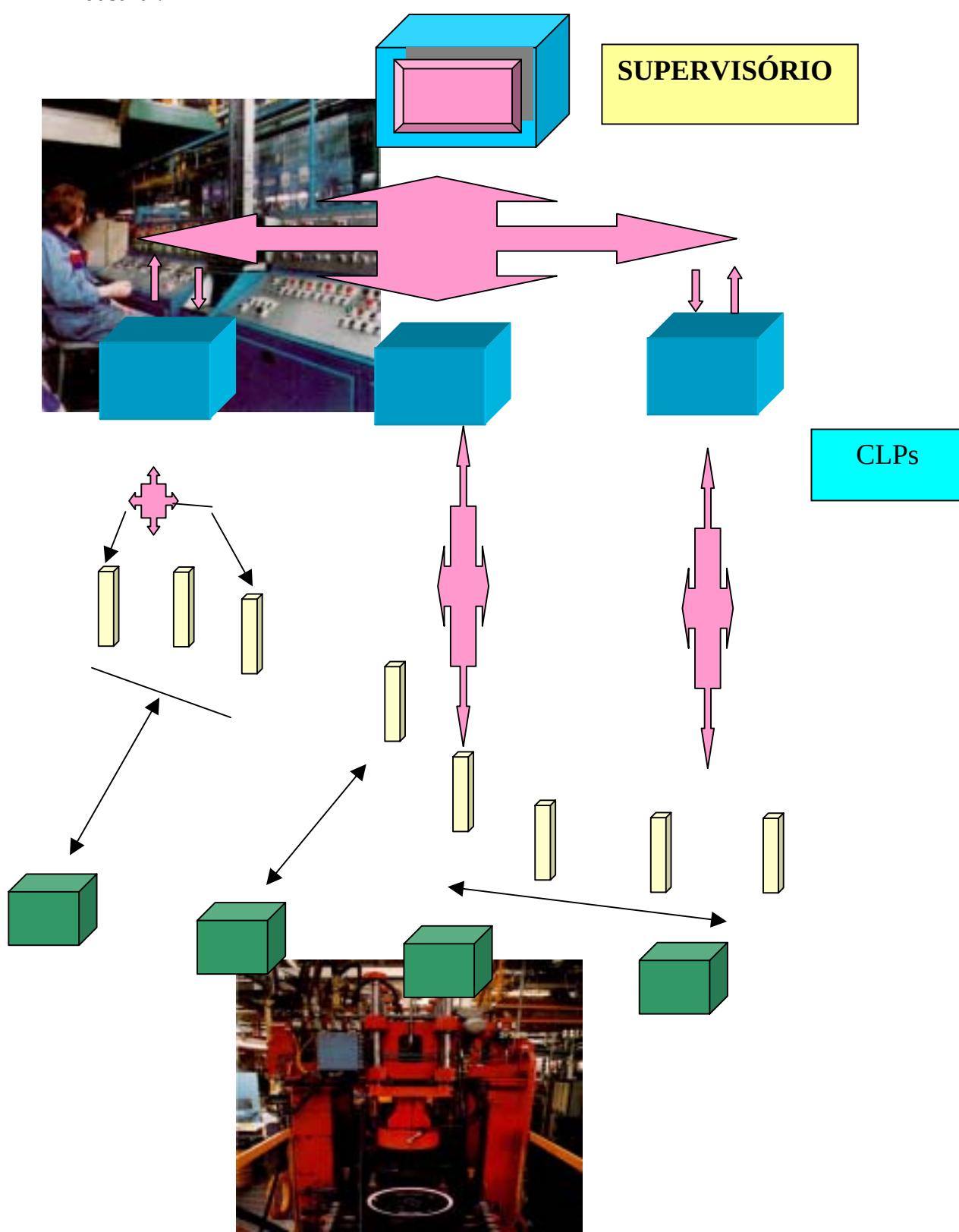


Fig. 9 Ilustrativo de uma automação implementada

10 Estrutura de Programa e do Sistema de Endereçamento do CLP SLC 500

Cada novo projeto de automação que envolva Controladores Programáveis deve, como sempre, obedecer às normas técnicas e, portanto, o engenheiro de automação deve inclusive adotar os procedimentos recomendados pela engenharia de software para que a gestão do projeto de automação possa ser eficiente e transparente. Assim sendo, o engenheiro de automação deve também, conhecer a estrutura de hardware e de software dos equipamentos empregados no projeto em questão. A estrutura do programa de um CLP da família SLC 500 obedece à seguinte arquitetura:

Cartões remotos

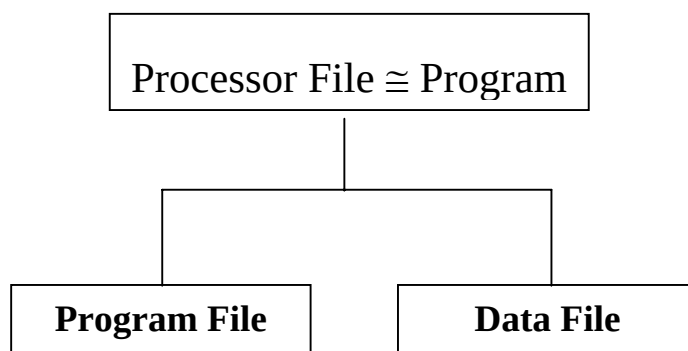


Fig. 10 Estrutura de programa no CLP SLC 500

Processor file: O *Processor file* (arquivo processador) é uma coletânea de arquivos de programa e de arquivos de dados criados sob um nome particular de **Processor file** ou **program**.

Cabe ao usuário definir o nome do **Processor file**. Sugere-se que a escolha do nome do **Processor files** seja a mais racional possível e que também, seja relacionada com o nome do projeto que se está trabalhando.

*Exemplo: Suponha que o controle de proteção da plataforma 25 de um campo petrolífero esteja sendo automatizado. Então um dos possíveis nomes dado ao **Processor file** deste projeto de automação da proteção da plataforma pode ser **PLAT_25_PROTEÇÃO_1***

Convém lembrar que a correta denominação deste nome adotada pelo engenheiro é fundamental para que a gestão do projeto possa ser realizada de forma eficiente.

O **Processor file** pode estar localizado: no terminal de trabalho do **RS Logix**, no hard disk, num disquete, no próprio processador do SLC ou num módulo de memória EEPROM ou UVPRM. O mesmo pode ser transferido, copiado, entre todas áreas descritas.

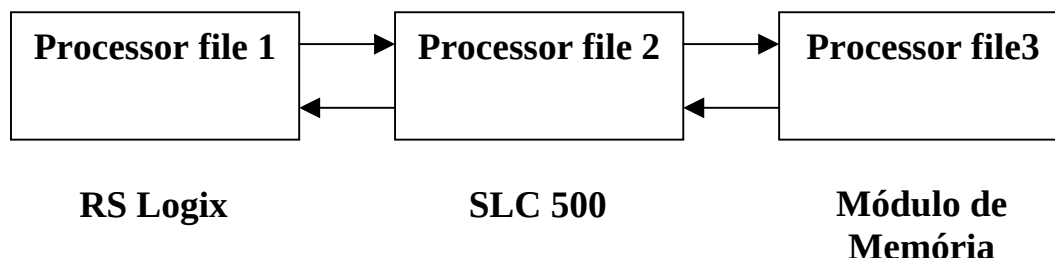


Fig. 11 Ilustrativo da localização de um Processor file

O **Processor file** conforme foi explicado é composto pelos: **Program Files** e do **Data Files**.

Program Files: Os **Program Files** ou os (Arquivos de Programas) são os arquivos que um contém as informações de controle e de configuração do hardware necessárias para que o CLP possa executar o seu programa. São compostos de três

arquivos básicos: *System Program*, *Main Ladder Program*, *Subroutine Ladder Program*. Sendo que os dois primeiros são gerados automaticamente a cada novo *program file* e o último é gerado opcionalmente pelo usuário.

- ***System Program / file 0*** (Programa de sistema / arquivo 0) – Este é o arquivo que contém as informações relacionadas ao sistema e as informações programadas pelo usuário, como: o tipo de processador, a configuração dos cartões de entrada e de saída, nome do arquivo processador, e senha.
- ***Main Ladder Program / file 2*** (Programa Ladder Principal / arquivo 2) – Este é o arquivo que contém as informações programadas pelo usuário que controlarão a operação da planta que está sendo automatizada. Em outras palavras é neste arquivo que o algoritmo do processo é implementado.
- ***Subroutine Ladder Program / file 3 - 255*** (Programa Ladder Sub-rotina) – Estes arquivos são criados opcionalmente pelo usuário, quando o mesmo julgar necessário que haja seqüências do algoritmo do processo “do programa ladder principal” que somente devam ser executadas quando eventos específicos ocorram. Desta forma o processador não desperdiça tempo de execução.

Data Files

Estes arquivos contêm os dados associados com as instruções que estão no arquivo Ladder de programa. Cada programa pode conter até 256 arquivos de dados. Estes arquivos são organizados pelo tipo de dados que eles contêm.

Os nove primeiros arquivos de dados numerados de 0 a 8 são criados automaticamente e são dos seguintes tipos.

- Saída / arquivo 0 – **Este arquivo é utilizado para armazenar os estados dos terminais de saída do controlador.**
- **Entrada / arquivo 1** – Este arquivo é utilizado para armazenar os estados dos terminais de entrada do controlador.
- **Status / arquivo 2** – Este arquivo é utilizado para armazenar as informações de operação do controlador como por exemplo “problemas” e operação de controle.
- **Bit / arquivo 3** – Este arquivo é utilizado para armazenar informações de bit internas.
- **Timer / arquivo 4** – Este arquivo é utilizado para armazenar as informações pertinentes dos temporizadores definidos pelo usuário.
- **Contador / arquivo 5** – Este arquivo é usado para armazenar o valor acumulado do contador e do preset, e também bits de status.
- **Controle / arquivo 6** – Este arquivo é utilizado para armazenar as informações pertinentes dos contadores definidos pelo usuário.
- **Integer / arquivo 7** – Este arquivo é utilizado para armazenar valores numéricos ou também as informações de bit.
- **Reservado / arquivo 8** – Este arquivo não é acessível para o usuário.
- **Definido pelo usuário / arquivos 9 – 255** – Estes arquivos são definidos pelo opcionalmente pelo usuário como sendo arquivos: bit, timer, contador, controle, e / ou armazenagem interna de dados.

10.1 Organização e Endereçamento dos Arquivos de Dados.

Os arquivos de dados contêm as informações de status associadas com as entradas e as saídas e com todas as outras instruções utilizadas no arquivo Ladder do programa principal ou nos arquivos de sub-rotinas.

Também podemos dizer que estes arquivos armazenam as informações relativas às operações do processador.

Os arquivos de dados estão localizados na memória do processador: A figura seguinte ilustra a seqüência dos nove primeiros arquivos de dados.

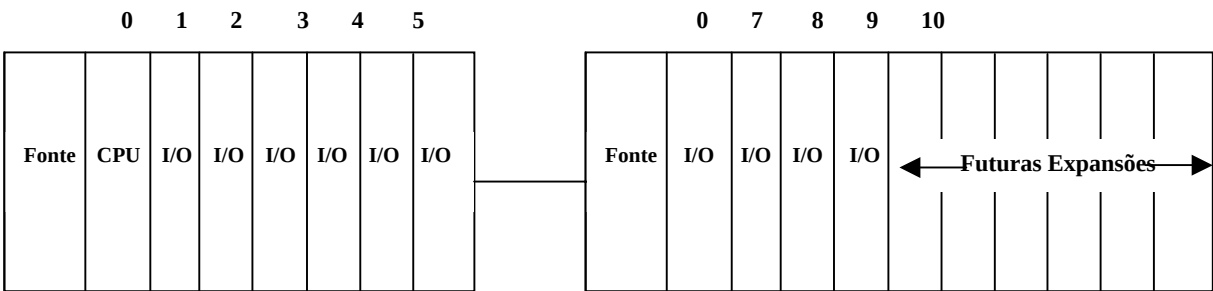
0	Saída de Imagem
1	Entrada de Imagem
2	Status
3	Bit
4	Timer
5	Counter
6	Control
7	Integer
8	Reservado
9	**
10 - 255	Bit, Timer, Counter, Control ou Integer, como for necessário.

** - O arquivo de dado número 9 pode ser usado para transferência de redes na rede DH-485.

10.2 Endereçamento de I/O para Controlador modular:

Com o controlador modular, o slot número 0 é reservado para o modulo do processador (CPU), portanto o endereço 0 do rack ou seja slot 0 é invalido para qualquer cartão de I/O.

A figura seguinte ilustra a configuração de uma arquitetura utilizando dois racks (um local, outro remoto).



As duas figuras seguintes ilustram trechos da memória do CLP que atendem á esta configuração.

Arquivo de dados 0 – Imagem de Saída

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	
Slot 1 / Saída (0 – 5)	-----INVALIDO-----							0								O:1
Slot 3 / Saída (0 – 15)	x															O:3
Slot 4 / Saída (0 – 7)	-----INVALIDO-----															O:4
Slot 5, Palavra 0 / Saída (0 – 15)																O:5
Slot 5, Palavra 1 / Saída (0 – 15)	X															O:5,1
Slot 9 / Saída (0 – 15)																O:9
Slot 10 / Saída (0 – 15)																O:10

Arquivo de dados 0 – Imagem de Entrada

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Slot 1 / Entrada (0 – 5)	-----INVALIDO-----							1	n								I:1
Slot 2, Palavra 0 / Entrada (0 – 15)																	I:3
Slot 2, Palavra 1 / Entrada (0 – 15)	-----INVALIDO-----											x					I:4
Slot 4 / Entrada (0 – 7)																	I:5
Slot 6 / Entrada (0 – 15)																	I:5,1
Slot 7 / Entrada (0 – 15)																	I:9
									x								I:10

Exemplos de bit:

O:3/15 Saída 15 do slot 3.

O:5/0 Saída 0 do slot 5.

O:10/11 Saída 11 do slot 10.

I:7/8 Entrada 8 do slot 7.

I:2.1/3 Entrada 3 do slot 2 palavra 1.

Exemplos de palavras:

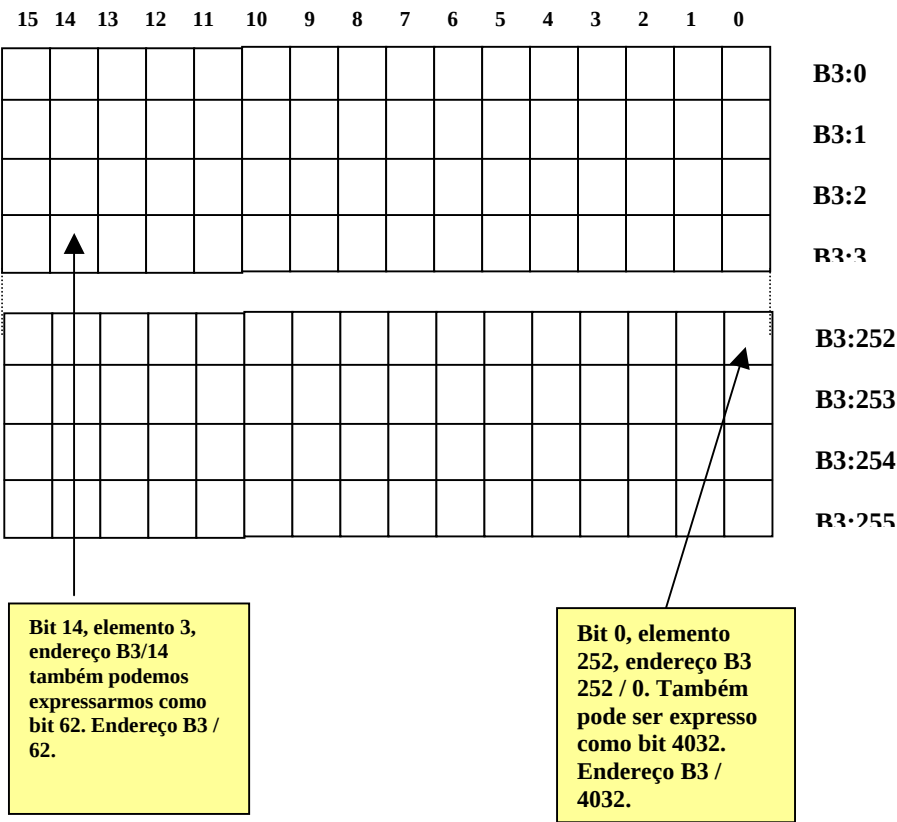
O:5 Palavra de saída 0 do slot 5

O:5.1 Palavra de saída 1 do slot 5

I:8 Palavra de entrada 0 slot 8

Arquivo de Dados número 3 – Bit

O arquivo 3 é o arquivo de bit, primeiramente usado para instrução de bit. O tamanho máximo deste arquivo é de 256 elementos de palavra, ou seja, um total de 4096 bits. Podemos endereçar um bit especificando o número do elemento (0 a 255) e o número de bit (0 a 15) dentro do elemento. Também se pode endereçar os bits numerando-os em sequência (0 a 4095). A figura seguinte ilustra a área de memória relativa ao arquivo de Bit.



Formatação	Explicação	
B f : e / b	B	
	f	
	:	
	e	
	/	
	b	

Arquivo de Dados número 4 – Temporizadores “Timers”

A instrução de temporização permite que o CLP possa computar e medir um determinado tempo a partir de um dado evento. Essa instrução é denominada de “timers”. A mesma emprega 3 palavras da memória do CLP conforme a seguinte estrutura. A palavra 0 é a palavra de controle, a palavra 1 armazena o valor pré-selecionado da temporizador e a palavra 2 armazena o valor acumulado: A figura seguinte ilustra a estrutura dessa instrução.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Palavra
EN			TT		DN		I						Uso Interno			0
Valor de Preset PRE																1
Valor Acumulado ACC																2

Bits de controle da palavra 0:

EN – *Bit 15 Enable* (bit 15 habilitado)

TT – *Bit 14 Timer Timing* (bit 14 temporizando)

DN – *Bit 13 Done*

A base de tempo pode ser definida pelo usuário. Pode ser de 0,01 seg. ou de 1 seg.

Exemplos: T4:0

Elemento 0, arquivo de timer 4.

T4: 0/15 ou T4:0 EN

T4: 0/14 ou T4:0 TT

T4: 0/13 ou T4:0 DN

T1: 0.1 ou T4:0 PRE

T4: 0.2 ou T4:0 ACC

T4: 0.1 / 0

T4: 0.2 / 0

Bit 0 do valor presente

Bit 0 do valor acumulado

Arquivo de Dado número 5 - Contadores

Os contadores são elementos de 3 palavras. A palavra 0 é a palavra de controle, a palavra 1 armazena o valor presente e a palavra 2 armazena o valor acumulado:

Elemento Contador

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Palavra	
EN			TT		DN		I					Uso Interno					0
Valor de Preset PRE																1	
Valor Acumulado ACC																2	

Bits de controle da palavra 0:

CU – Contagem crescente habilitada.

CD – Contagem decrescente habilitada.

DN – Bit de DONE.

OV – Bit de OVERFLOW .

UN – Bit de UNDERFLOW.

UA – Valor Acumulado atualizado.

Exemplos: C5:0

Elemento 0, arquivo CONTADOR 5.

C5: 0/15 ou C5:0 CU

C5: 0/14 ou C5:0 CD

C5: 0/13 ou C5:0 DN

C5: 0/12 ou C5:0 OV

C5: 0/11 ou C5:0 UN

C5: 0/10 ou C5:0 UA

C1: 0.1 ou C5:0 PRE

C5: 0.2 ou C5:0 ACC

C5: 0.1 / 0

C5: 0.2 / 0

Bit 0 do valor presente

Bit 0 do valor acumulado

11. Bibliografia:

R. W. Lewis “Programming industrial control systems using IEC 1131-3” – 1995.
G. Michell “Programmable Logic Controllers” Architecture and Applications” John Wiley & Sons Ltd. England 1990.
Allen-Bradley “Micro Mentor. Entendendo e utilizando os microcontroladores Programáveis” 1999.
P. L. Castrucci, C. C. Moraes ” Engenharia de Automação Industrial” LTC Livros Técnicos e Científicos Editores S.A. 2001.
F. Natale “Automação Industrial” Érica 1995.
Manuais – Rockwell Automation
Manuais – AEG Schneider Automation
C. C. Moraes, E. Senger, S. L. Pereira – “Laboratório de Automação” PEA – EPUSP 1995.